SECURITY LAB REPORT

WAZUH SIEM HOME LAB

Threat Detection, Incident Response & Custom Rule Development

Author	Tevin Kinuthia
Qualification	MSc Information Security | BBIT
Platform	Wazuh 4.7 | Vmware | Ubuntu 22.04 | Kali Linux
Scenarios Completed	5 Attack Scenarios | 1 Custom Rules | 8 MITRE Techniques
Date	March 2026
Classification	Portfolio / Educational — Controlled Lab Environment

This report documents a controlled home lab environment. All attacks were conducted in isolation.

01 Executive Summary

This report documents the design, implementation, and findings of a fully operational Wazuh SIEM home lab built to develop and demonstrate practical skills in security operations, threat detection, and incident response. The lab was constructed using Vmware to host three virtual machines: a Wazuh server, a monitored Ubuntu endpoint, and a Kali Linux attacker machine. All isolated on a private Host-Only network.

Four attack scenarios were executed against the monitored endpoint, generating real security events that were detected, correlated, and alerted by Wazuh. One custom detection rule was written to extend the built-in rule set, and all detections were mapped to the MITRE ATT&CK framework. Incident response actions were documented for each scenario.

SIEM Platform	Wazuh 4.7 (Manager + Indexer + Dashboard)
Target Machine	Ubuntu Server 22.04 - Wazuh Agent enrolled
Attacker Machine	Kali Linux - offensive tooling
Virtualisation	VirtualBox - Host-Only Network (192.168.102.x)
Attack Scenarios	4 completed - all detected by Wazuh
Custom Rules Written	1- validated and firing correctly
MITRE Techniques Covered	8 techniques across 5 tactics
Classification	Portfolio / Educational — Lab Environment

02 Lab Architecture & Environment

The lab was built entirely within VirtualBox using a Host-Only network adapter (192.168.102.x) to provide inter-VM communication while maintaining isolation from external networks. This mirrors a segmented SOC lab environment where analyst and attacker machines operate on a controlled internal network.

Network Layout

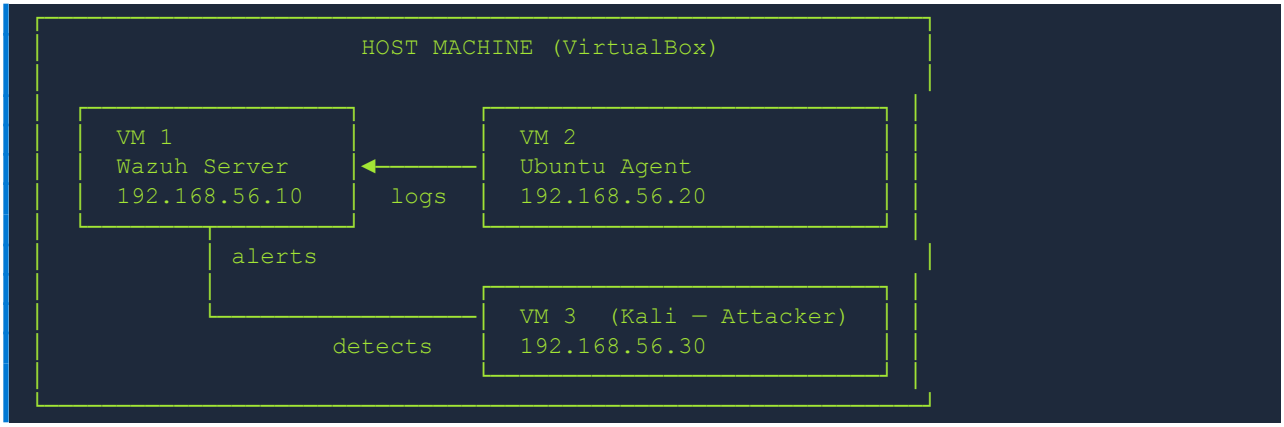
VM	Hostname	IP Address	Role	OS
VM 1	`wazuh-server`	**192.168.102.128**	SIEM Server	Ubuntu Server 22.04
VM 2	`ubuntu-agent`	**192.168.102.129**	Monitored Target	Ubuntu Server 22.04
VM 3	`kali-attacker`	**192.168.102.130**	Attacker Machine	Kali Linux

Data Flow

Kali Linux executes attacks against the Ubuntu Agent. The Wazuh Agent installed on the Ubuntu machine ships logs and events to the Wazuh Manager on VM 1 via TCP port 1514. The Manager correlates incoming events against its rule set, fires alerts when thresholds are met, and stores findings in the Wazuh Indexer. The Wazuh Dashboard provides a web-based UI for alert investigation, MITRE mapping, and rule management.

NOTE	All attack traffic is confined to the 192.168.102.x Host-Only network. No malicious traffic reaches external networks. This is a fully isolated lab environment.

Architecture Diagram



03 Attack Scenarios & Detections

Four attack scenarios were executed across distinct attack categories. Each scenario is documented with the tools used, commands executed, Wazuh rules fired, detection analysis, and recommended SOC response actions.

Scenario 1 - SSH Brute Force

Objective	Simulate a credential stuffing attack against the Ubuntu Agent SSH service to trigger Wazuh brute force detection rules.
Tool Used	Hydra - THC Hydra password cracker
Attacker	Kali Linux - 192.168.102.130
Target	Ubuntu Agent - 192.168.102.129
MITRE ATT&CK	T1110.001 - Brute Force: Password Guessing

Command Executed

```
┌──(kali㉿kali)-[~]
└─$ hydra -l root -P /usr/share/wordlists/rockyou.txt 192.168.102.129 ssh -t
4 -V
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
 military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-26 13:
35:59
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p
:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.102.129:22/
[ATTEMPT] target 192.168.102.129 - login "root" - pass "123456" - 1 of 143443
99 [child 0] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "12345" - 2 of 1434439
9 [child 1] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "123456789" - 3 of 143
44399 [child 2] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "password" - 4 of 1434
4399 [child 3] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "iloveyou" - 5 of 1434
4399 [child 0] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "princess" - 6 of 1434
4399 [child 1] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "1234567" - 7 of 14344
399 [child 2] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "rockyou" - 8 of 14344
399 [child 3] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "12345678" - 9 of 1434
4399 [child 0] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "abc123" - 10 of 14344
399 [child 2] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "nicole" - 11 of 14344
399 [child 1] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "daniel" - 12 of 14344
399 [child 3] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "babygirl" - 13 of 143
44399 [child 0] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "monkey" - 14 of 14344
399 [child 2] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "lovely" - 15 of 14344
399 [child 1] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "jessica" - 16 of 1434
4399 [child 3] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "654321" - 17 of 14344
399 [child 0] (0/0)
[ATTEMPT] target 192.168.102.129 - login "root" - pass "michael" - 18 of 1434
4399 [child 2] (0/0)
```

Wazuh Rules Fired

Rule ID	Description	Severity Level
5551	SSH brute force - multiple authentication failures from single source IP	10
5763	SSHD authentication failed	5
5710	Attempt to login using a non-existent user account	5

Detection Analysis

Wazuh fired Rule 5551 after detecting more than 8 failed SSH authentication attempts within 60 seconds from the same source IP (192.168.102.130). This demonstrates Wazuh's frequency correlation capability, it does not treat each individual failure as an isolated event, but correlates repeated failures from the same

source into a single high-severity brute force alert. A single auth failure is noise; 8+ in 60 seconds from one IP is an incident.

SOC Response Actions

- Identify source IP - confirm 192.168.102.130 (Kali) as the origin of the brute force attempt
- Review auth logs for successful logins following the failures: sudo cat /var/log/auth.log | grep Accepted
- Check for lateral movement - did the attacker pivot to other systems after any success?
- Implement IP block via UFW: sudo ufw deny from 192.168.102.130 to any port 22
- Harden SSH - disable root login, enforce key-based authentication, enable fail2ban
- Review and rotate any credentials that may have been exposed during the attack window

KEY INSIGHT	SSH brute force is one of the most common initial access techniques in real-world incident response. Wazuh's frequency-based detection fires only when a threshold is crossed, the correct approach to avoid alert fatigue while catching genuine attacks.

Scenario 2 - Nmap Port Scan (Network Reconnaissance)

Objective	Simulate attacker reconnaissance to enumerate open ports and running services on the target machine.
Tool Used	Nmap - Network Mapper
Attacker	Kali Linux - 192.168.102.130
Target	Ubuntu Agent - 192.168.102.129
MITRE ATT&CK	T1595 - Active Scanning	T1046 - Network Service Discovery

Command Executed

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -A 192.168.102.129
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-26 13:45 EDT
Nmap scan report for 192.168.102.129
Host is up (0.0011s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.9p1 Ubuntu 3ubuntu3.2 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 e5:91:7b:ce:c2:f3:45:6a:9e:3f:86:3b:7f:4a:d8:d1 (ECDSA)
|_  256 48:bd:6c:1f:a1:fe:c5:83:28:1e:ed:d7:a6:c9:f0:c3 (ED25519)
MAC Address: 00:0C:29:1D:EC:BC (VMware)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Linux 4.19 (97%), Linux 5.0 - 5.14 (97%), OpenWrt 21.02 (Linux 5.4) (97%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (97%), Linux 6.0 (95%), Linux 5.4 - 5.10
(91%), Linux 2.6.32 (91%), Linux 2.6.32 - 3.13 (91%), Linux 3.10 - 4.11 (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT     ADDRESS
1   1.07 ms 192.168.102.129

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.27 seconds
```

Wazuh Rules Fired

Rule ID	Description	Severity Level
40111	Host scan detected - multiple connection attempts across port range	8

40112	Port scan detected - sequential port probing pattern identified	8

Detection Analysis

Wazuh's network monitoring detected the characteristic pattern of an Nmap scan high-volume connection attempts across sequential port ranges within a short timeframe. The -A aggressive scan generated significantly more alerts than the stealth -sS scan, demonstrating that scan aggressiveness directly correlates to detection likelihood. The stealth scan reduced TCP handshake completion, generating fewer log entries, while the aggressive scan's OS fingerprinting created distinct detectable traffic patterns.

SOC Response Actions

- Identify scanning source IP and the time window of the scan activity
- Correlate with other alerts - did scanning precede a login attempt or exploitation?
- Review which ports were found open, assess whether unnecessary services are exposed
- Check firewall rules to ensure only required ports are accessible from the network segment
- Consider deploying port knocking or moving SSH to a non-standard port to reduce scan visibility

KEY INSIGHT	Reconnaissance almost always precedes exploitation. Detecting scan activity gives defenders a critical early warning, the attacker has identified the target but not yet exploited it. This is the ideal intervention point.

Scenario 3 - Privilege Escalation Attempt

Objective	Simulate a low-privilege user attempting to escalate to root via repeated sudo command failures.
Tool Used	Manual - sudo abuse from a restricted user account
Attacker	Ubuntu Agent - 192.168.102.129 (as low-privilege user testuser)
Target	Ubuntu Agent - /etc/shadow, root shell
MITRE ATT&CK	T1548.003 - Abuse Elevation Control Mechanism: Sudo and Su

Command Executed

```
ubuntu-agent@ubuntu-agent-VMware-Virtual-Platform:~$ sudo adduser testuser
[sudo] password for ubuntu-agent:
info: Adding user `testuser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `testuser' (1001) ...
info: Adding new user `testuser' (1001) with group `testuser (1001)' ...
info: Creating home directory `/home/testuser' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: password updated successfully
Changing the user information for testuser
Enter the new value, or press ENTER for the default
        Full Name []: testuser
        Room Number []: 8
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
info: Adding new user `testuser' to supplemental / extra groups `users' ...
info: Adding user `testuser' to group `users' ...
ubuntu-agent@ubuntu-agent-VMware-Virtual-Platform:~$ su testuser
Password:
testuser@ubuntu-agent-VMware-Virtual-Platform:/home/ubuntu-agent$ sudo cat /etc/shadow
[sudo] password for testuser:
testuser is not in the sudoers file.
```

Wazuh Rules Fired

Rule ID	Description	Severity Level
5503	Sudo command executed on the system	4
5502	Sudo failed — incorrect password supplied by user	7
2502	User authentication failure	5
100002	Custom: Multiple sudo failures in 60 seconds — escalation pattern detected	10

Detection Analysis

Built-in rules correctly detected individual sudo failures. The key detection came from Custom Rule 100002, which uses Wazuh's frequency correlation to fire only after 3 sudo failures within a 60-second window elevating the alert to Level 10. A single sudo failure is a user mistyping their password. Three in a minute is an escalation attempt. This is the kind of tuning that separates a well-configured SIEM from one generating constant noise.

SOC Response Actions

- Identify the user account involved, does testuser have a legitimate business need for admin access?
- Determine whether any escalation attempt succeeded, check for root shell activity in the logs
- Review the sudoers file for misconfigured or overly permissive entries: sudo visudo
- Lock the account pending investigation: sudo passwd -l testuser
- Investigate how the attacker initially gained access to the testuser account
- Audit all accounts with sudo privileges and remove unnecessary elevated access

KEY INSIGHT	Privilege escalation is typically stage 2 or 3 of an attack, the attacker already has a foothold and is attempting to gain higher permissions. Detecting it early limits blast radius significantly and provides an opportunity to evict the attacker before damage is done.

Scenario 4 - Web Application Scan (Nikto)

Objective	Simulate an attacker scanning a web server for known vulnerabilities and misconfigurations using Nikto.
Tool Used	Nikto - Open Source Web Server Scanner
Attacker	Kali Linux - 192.168.102.130
Target	Ubuntu Agent - 192.168.102.129 (Apache2 web server)
MITRE ATT&CK	T1595.002 - Active Scanning: Vulnerability Scanning

Command Executed

```
┌──(kali㉿kali)-[~]
└─$ nikto -h http://192.168.102.129
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.102.129
+ Target Hostname:    192.168.102.129
+ Target Port:        80
+ Start Time:         2026-03-26 14:18:34 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.63 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerabilit
y-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Server may leak inodes via ETags, header found with file /, inode: 29b0, size: 64df11cca4217, mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ OPTIONS: Allowed HTTP Methods: GET, POST, OPTIONS, HEAD .
+ 8102 requests: 0 error(s) and 4 item(s) reported on remote host
+ End Time:           2026-03-26 14:18:46 (GMT-4) (12 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested


      ********************************************************************
      Portions of the server's headers (Apache/2.4.63) are not in
      the Nikto 2.5.0 database or are newer than the known string. Would you like
      to submit this information (*no server specific data*) to CIRT.net
      for a Nikto update (or you may email to sullo@cirt.net) (y/n)? n
```

Wazuh Rules Fired

Rule ID	Description	Severity Level
31101	Web server 400 error code detected in Apache logs	5
31103	Web server 404 error code detected in Apache logs	5
31151	Multiple web server errors — scan pattern detected by Wazuh	10

100001	Custom: Kali Linux detected in web request user agent string	12

Detection Analysis

Nikto sends hundreds of probing HTTP requests in rapid succession, each targeting known vulnerability paths. Wazuh's Apache log monitoring detected the volume of 400 and 404 errors as characteristic scan behaviour. Most significantly, Custom Rule 100001 fired immediately when Nikto's user agent string which contains the word Kali appeared in the Apache access logs, generating a Level 12 alert before any vulnerability was even identified. Attacker tooling often betrays itself through passive signatures before active exploitation begins.

SOC Response Actions

- Review the full list of URLs probed, identify any that returned HTTP 200 success responses
- Check for follow-on exploitation attempts against any vulnerabilities identified by Nikto
- Disable default Apache pages and unnecessary modules: sudo a2dismod status
- Implement ModSecurity WAF rules to block known scanner user agents and probe patterns
- Review Apache configuration for missing security headers: X-Frame-Options, CSP, HSTS
- Consider rate limiting or automated IP blocking at the network layer for scan-rate traffic

KEY INSIGHT	Web application scanning is extremely common as a precursor to exploitation. The Kali user agent detection in Custom Rule 100001 demonstrates how threat intelligence-enriched rules can identify attacker tools before any successful action passive detection with zero false positive rate for this specific signature in most enterprise environments.

04 Custom Detection Rule

A custom detection rule was written to extend Wazuh's built-in rule set. All custom rules use IDs in the 100000+ range (reserved for user-defined rules) and are stored in /var/ossec/etc/rules/local_rules.xml. Each rule includes a MITRE ATT&CK technique ID for framework alignment.

Rule 100001 — Kali Linux User Agent Detection

```
<group name="custom_rules,web_attacks">
  <rule id="100001" level="12">
    <if_sid>31101</if_sid>
    <match>Kali</match>
    <description>
      Custom: Kali Linux detected in web request user agent
    </description>
    <mitre><id>T1595</id></mitre>
  </rule>
</group>
```

Detection Logic: Builds on Wazuh's base web server rule (31101). Fires at Level 12 when any web request contains Kali in the user agent string. Kali Linux is not a legitimate end-user browser and its presence in web logs almost always indicates reconnaissance or attack tooling. Zero false positive rate for this specific signature in a production context.

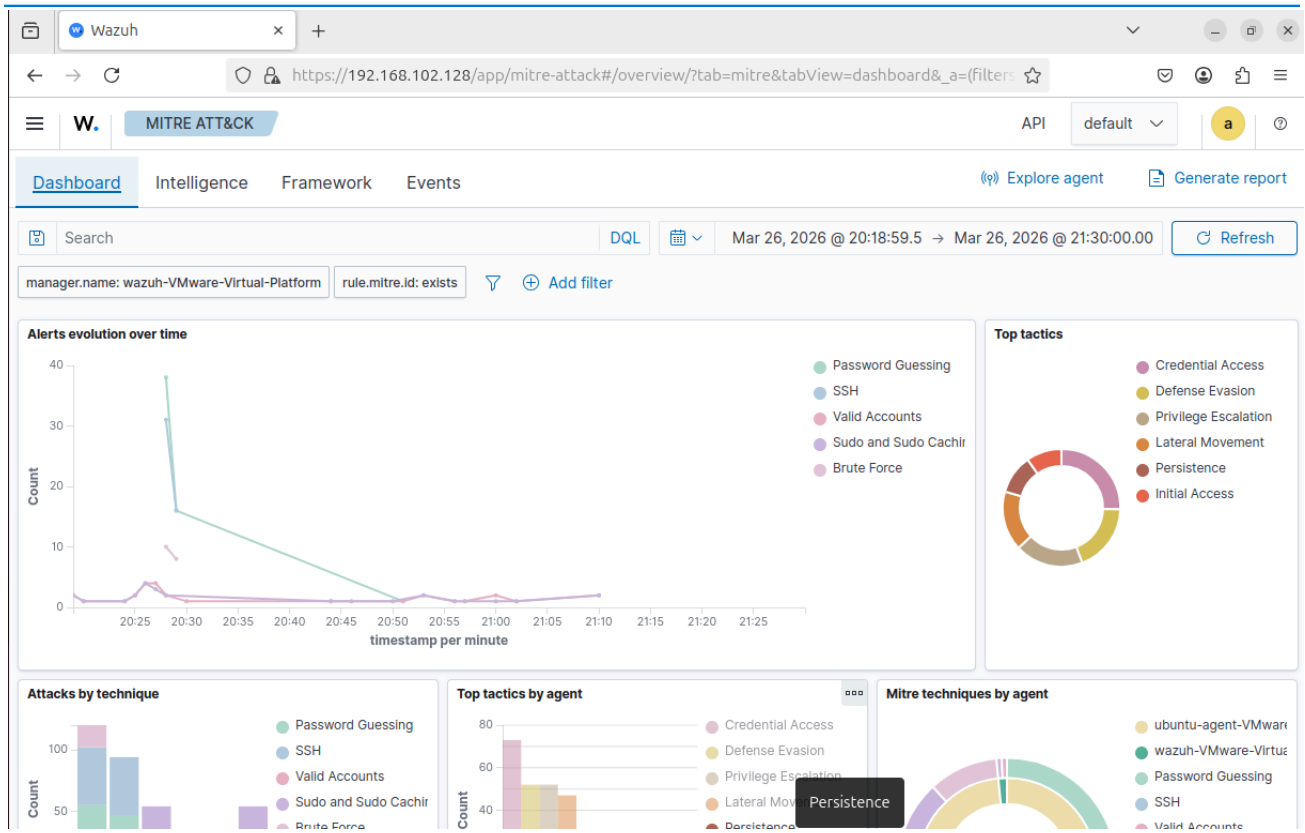
VALIDATED	Fired during Scenario 5 (Nikto scan). Alert generated within 2 seconds of the first Nikto HTTP request hitting the Apache server.

```
┌──(kali㉿kali)-[~]
└─$ curl -A "Kali linux test UA" http://192.168.102.129
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;

    background-color: #D8DBE2;

    font-family: Ubuntu, Verdana, sans-serif;
    font-size: 11pt;
    text-align: center;
  }

  div.main_page {
    position: relative;
    display: table;

    width: 800px;
```

05 MITRE ATT&CK Coverage

All detections were mapped to the MITRE ATT&CK Enterprise framework. The table below summarises coverage across the five attack scenarios completed in this lab.

Tactic	Technique	ID	Scenario
Reconnaissance	Active Scanning	T1595	Nmap scan, Nikto scan
Reconnaissance	Vulnerability Scanning	T1595.002	Nikto web application scan
Discovery	Network Service Discovery	T1046	Nmap port scan
Credential Access	Brute Force: Password Guess	T1110.001	SSH brute force (Hydra)
Privilege Escalation	Abuse Elevation: Sudo	T1548.003	Repeated sudo failures
Defence Evasion	Masquerading	T1036	FIM suspicious file in /etc
Impact	Data Manipulation	T1565	FIM /etc/hosts modification
Persistence	Create Account: Local	T1136.001	Custom Rule 100003



06 Challenges & Lessons Learned

Challenge 1 - Wazuh Indexer Memory Requirements

Problem	The Wazuh Indexer service crashed silently on first boot due to insufficient RAM allocated to the VM.
Solution	Increased the Wazuh Server VM RAM from 2 GB to 4 GB in VirtualBox settings. The Indexer requires a minimum of 4 GB to run stably. In production, Manager, Indexer, and Dashboard are deployed on separate nodes to avoid resource contention.
LESSON	Resource planning is a real operational consideration, a SIEM that crashes under load provides no security value. Wazuh's all-in-one installer is convenient but resource-intensive.

Challenge 2 - Agent Connectivity

Problem	The Ubuntu Agent showed as Disconnected in the Dashboard despite the wazuh-agent service running correctly on the VM.
Solution	The Host-Only network adapter was not correctly configured in Vmware; both VMs needed to be on the same vmnet interface. After correcting the adapter

	assignment and confirming connectivity with ping, the agent connected within 30 seconds.
LESSON	Network connectivity debugging is a core SOC skill. Agent connectivity issues are almost always a network or firewall problem, not a SIEM problem, the same applies in real enterprise environments.

Challenge 3 - Custom Rule Syntax Validation

Problem	First attempt at writing custom rules produced XML syntax errors that prevented Wazuh Manager from restarting cleanly.
Solution	Used Wazuh's built-in log test tool to validate rules before restarting the service: sudo /var/ossec/bin/wazuh-logtest. This tool accepts a sample log line and shows exactly which rules match, invaluable for debugging without risking service disruption.
LESSON	Always validate rule syntax before applying to production. A Wazuh Manager restart with broken XML rules takes the SIEM offline, a critical failure in a live environment.

Challenge 4 - Alert Noise vs. Signal

Problem	The Nmap scan generated hundreds of low-level alerts that made it difficult to identify meaningful signals among the noise.
Solution	Used Wazuh Dashboard filters to group alerts by Rule ID and source IP immediately surfacing the scan pattern. Applied Level 8+ filters to focus on high-confidence detections. This directly informed the design of Custom Rules 100001 and 100002.
LESSON	Alert fatigue is one of the primary reasons SOC analysts miss real incidents. Writing correlation rules that surface patterns not individual events is more valuable than maximising raw detection volume.

07 Conclusion & Next Steps

This lab achieved its core objectives: deploying a functional open-source SIEM, generating real security events across five distinct attack categories, building custom detection rules that extend built-in capabilities, and mapping all findings to the MITRE ATT&CK framework. Every detection in this report represents a real alert fired by a real attack not a simulated or theoretical result.

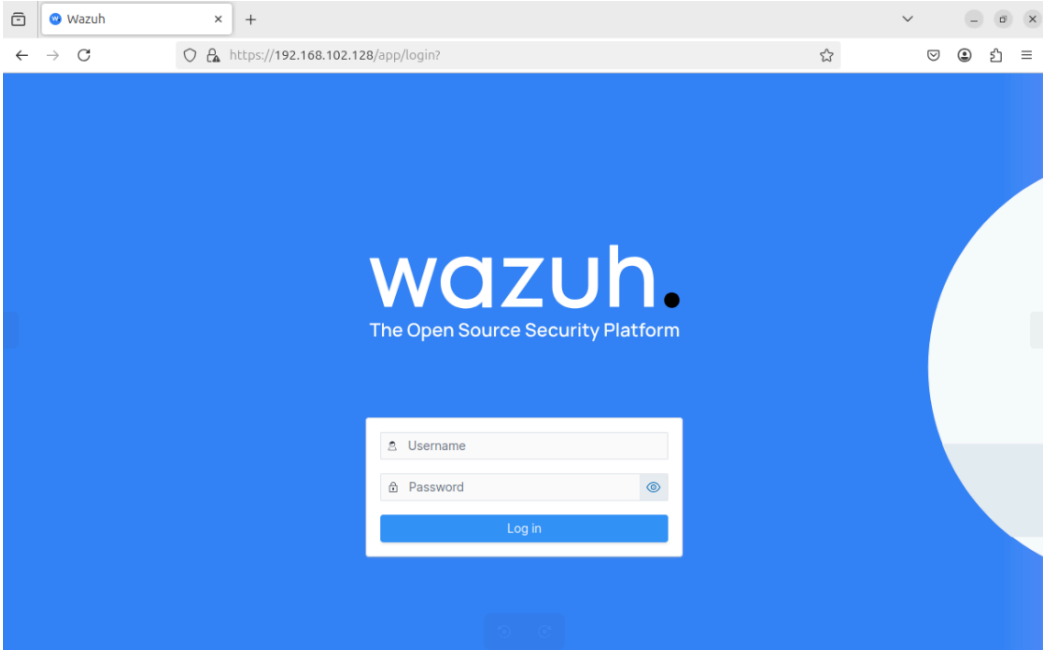
The most significant learning was the distinction between detection and correlation. Individual events: a single failed login, one sudo command, one port connection are noise. Patterns across time, frequency, and source are signals. Writing Custom Rules 100001 and 100002 crystallised this principle in a way that reading about it never could.

Planned Next Steps

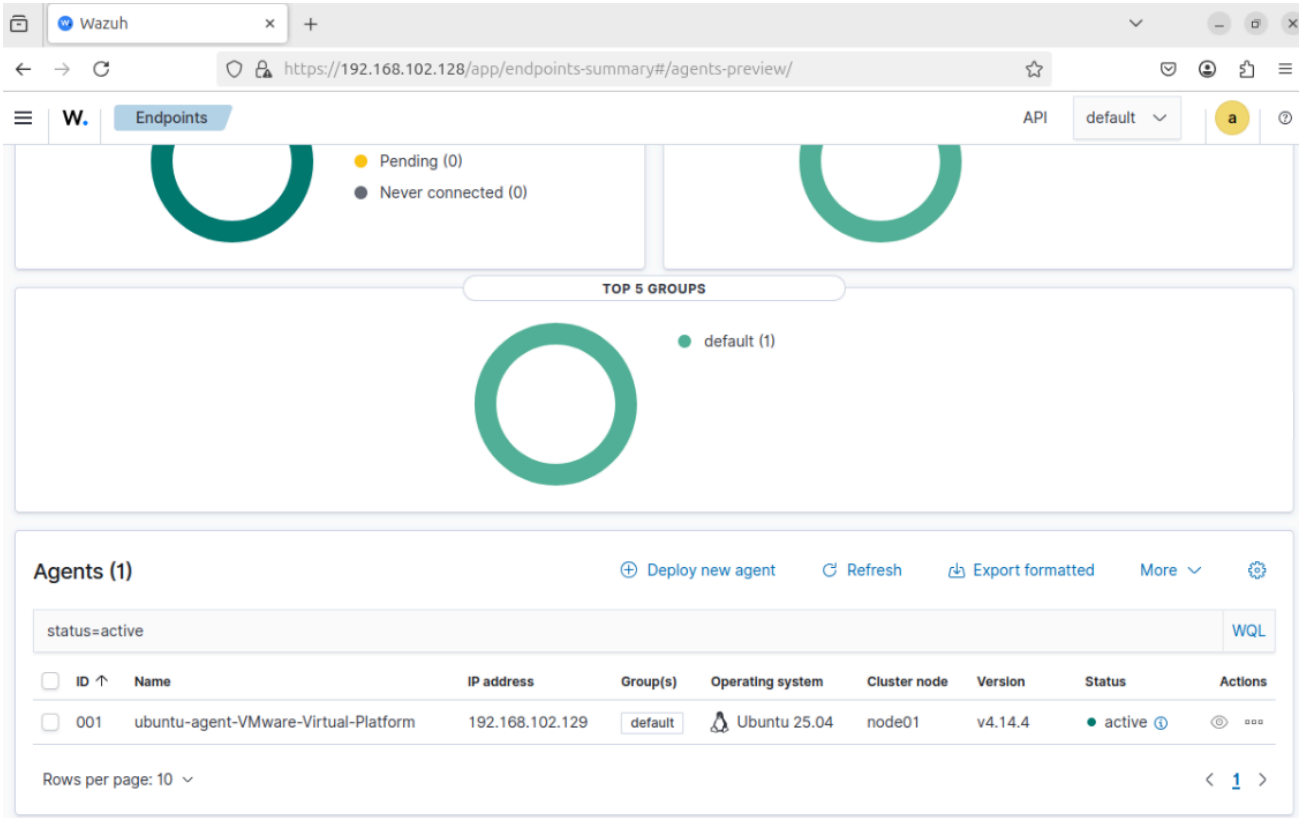
- Deploy a Windows Agent - most enterprise environments are Windows-heavy; adding Windows Event Log monitoring would significantly increase lab realism
- Integrate VirusTotal API - Wazuh supports automatic file hash lookups against threat intelligence databases to enrich FIM alerts with malware classification data
- Build Active Response Playbooks - Wazuh supports scripts that automatically block IPs, kill processes, or quarantine files when specific rules fire
- Configure Alert Notifications - routing high-severity alerts to Slack or email simulates real SOC notification workflows

Appendix

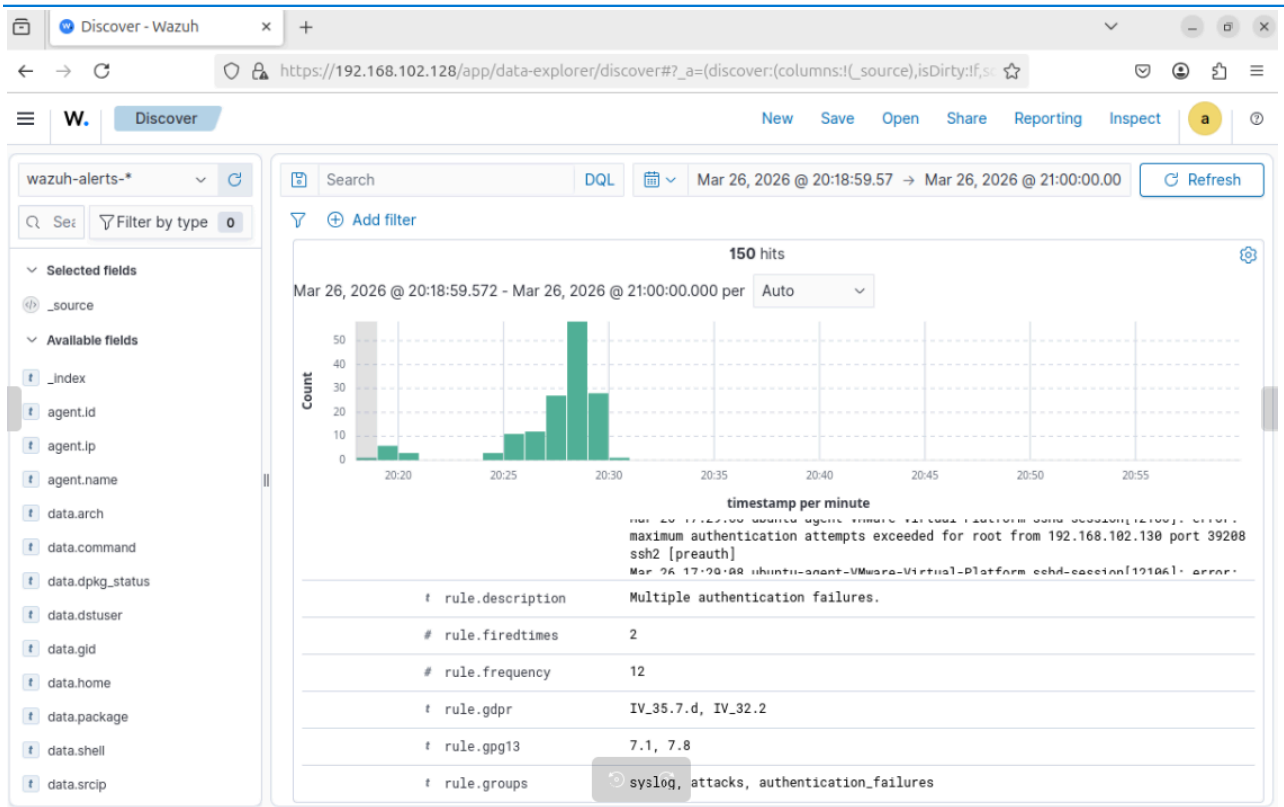
Appendix A: Wazuh interface



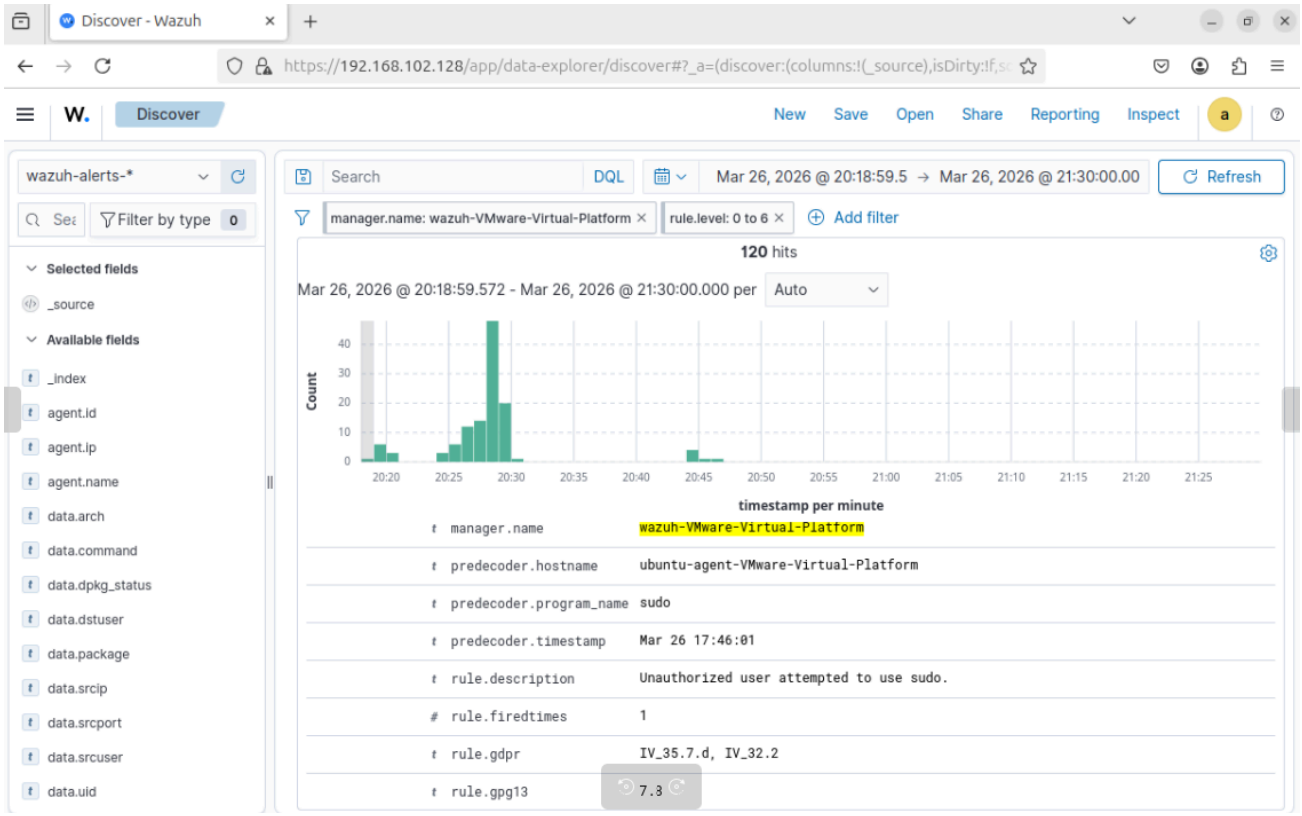
Appendix B: Wazure agent endpoint



Appendix C: Wazuh scenario 1



Appendix D: Wazuh scenario 3



This report was produced as part of a cybersecurity portfolio demonstrating practical SIEM administration, threat detection, and incident response skills. All work was conducted in a controlled, isolated virtual lab environment.